THE OPEN SOURCE AUDIT

A Capstone Project Report

Subject: Linux Kernel

Student Name:

Abhishek Choudhary

Registration No:

24BCE11056

Course:

Open Source Software (NGMC)

Software Chosen: Linux Kernel | License: GPL v2

INTRODUCTION

Down near the bottom of things, there's a piece called the Linux Kernel. Most web servers rely on it, so do clouds, phones, even giant computers crunching numbers nonstop. Cars use bits of it too, along with those little boxes routing internet traffic around homes. Hidden from view, it stays out of sight for nearly everyone touching devices every day. Never makes noise about being there. Just keeps going, doing its job beneath layers others never see.

From time to time, someone takes a close look at the Linux Kernel - not only as code - but as something made by people. Instead of listing functions, this one digs into reasons behind its creation. Shaped by certain beliefs, those ideas show up in every line. Its license isn't just legal text - it sets boundaries and grants freedom, depending on who reads it. Over years, ripples spread outward - tools appeared, groups formed, ways of working changed. Growth didn't stop; others kept building beside and beyond it.

Four sections make up this document. Starting with origins, Section A looks into how the kernel began, weaving in thoughts on ethics and openness behind shared code. Moving along, part B shows where the Linux Kernel sits inside a working system - its folders, add-on pieces, leaving room for updates too. Not far beyond, Section C traces the wider world of free software around Linux, showing links both ways: what feeds it, what grows from it. Ending here, Part D lines up Linux beside closed systems, measuring them closely before landing on practical outcomes.

Besides the written parts, five shell scripts were built during this project. Each one shows real-world use of Linux command line tools while tying into the open-source system under review. Running them took place in a Unix-style terminal, where every step was checked carefully. Detailed records of both code and results appear later in the document. The last part holds full documentation of each test outcome.

Something interesting happens when code becomes a collective effort. The Linux Kernel stands out, not only due to its complexity, yet more so for how it emerged - built quietly, piece by piece, through countless hands across continents. People who never met shaped it over time. Their work together shows a different side of technology - not just tools, but trust forming in the background. What starts as lines

in a file turns into shared ownership, spread globally. This kind of creation doesn’t shout; it accumulates.

Part A — Origin and Philosophy

A1. The Problem the Linux Kernel Was Created to Solve

Picture 1991 first, if you want to grasp how the Linux Kernel came to be. Back then, computers worked in ways most now find hard to imagine.

Back when computers meant real work, one name stood out: Unix. Research labs used it. So did colleges. Businesses relied on it too. Strong performance met smart design - a rare combo that actually worked. Yet cost climbed high, not only for gear but access rights. Ownership stayed locked down by companies. Running it by the rules meant paying up front. Without support from a university or lab, most students found the price impossible to meet.

A project emerged to tackle this exact problem. Back in the 1980s, Andrew Tanenbaum built MINIX - a compact, Unix-style OS meant for learning. Its clean layout, along with included source code, stood out during an era when such access was rare. Yet despite its strengths, it carried a key restriction. Because it focused on classroom use, full functionality took a back seat. Minimalism wasn’t accidental; it shaped the system's entire purpose. His okay needed before any new parts arrived. The goal wasn’t just about stretching the core.

This created an opening. An environment existed where learning happened without creating. Meanwhile, across the scene stood the GNU Project - Richard Stallman’s push for a fully open Unix-style system - delivering key components such as GCC, Emacs, and the Bash shell, yet lacking one core piece: a functional kernel. Development on the GNU Hurd limped along, far from practical readiness.

Linus Torvalds showed up right there. Back in August 1991, he was two decades old, studying computing at Helsinki's university. MINIX had caught his attention earlier; yet its limits frustrated him, so he started crafting a kernel on his own - only meant to explore how his 386 machine worked, nothing grand. He later mentioned that spark came purely from curiosity. A message posted to comp.os.minix called the effort "just a hobby," adding it would never reach GNU’s

scale or polish. That note carried no weight at first. Yet things unfolded differently than expected

That final piece? He didn't get it right.

Back in September 1991, a first tiny version appeared - just 0.01 - named Linux by its creator. Even more crucial? The source code came along with it, open for anyone. Folks started reading, testing, changing it almost right away. Within weeks, fixes poured in from programmers across different countries. Come 1992, everything shifted when the core switched to the GNU license. That move locked in freedom through legal tools built years earlier.

By chance, Torvalds ended up bridging the space where GNU tools met an actual running kernel. Together - Linux paired with those tools - formed a full operating system that worked and cost nothing. When he chose to share the source openly, inviting input from all comers, everything shifted in size and scope.

What started as one person wanting a usable, no-cost alternative to Unix became something wider. Linus Torvalds built the Linux Kernel so he could tinker freely on his computer. Yet its impact grew far beyond individual tinkering once others got involved. Open availability of the code allowed anyone to view, change, or share it without gatekeepers. Over time, what began in solitude turned into a collective effort across continents. Free from fees or permission rules, the kernel reached schools, businesses, homes. Its value wasn't just technical - it reshaped how people collaborate around software.

A2. The License — What It Actually Says

A license called GPL v2 covers the Linux Kernel. What that actually allows comes clear when looking at four key rights tied to free software. These were laid out by Richard Stallman along with the GNU Project.

Folks might run software however they see fit - no restrictions there. Look under the hood, tinker with code, that sort of thing comes next. Sharing duplicates? Absolutely allowed, pass them around freely. Changed something yourself? Hand those updates out just like the original. Freedoms stack up to turn users into active contributors, not just silent watchers. What a program actually does? That becomes visible, open to study. Changes fit their needs when someone tweaks the lines

themselves. Sharing slips copies into new hands without barriers. Building happens naturally, one tweak at a time, layer after layer.

Freedom number one, two, three, plus four - all covered by GPL v2. Yet here's the twist: share an altered form, then the source changes must also be shared just as openly. That rule? It flips copyright on its head - uses it to lock in liberty instead of locking users out. As copies move from person to person, each keeps those rights intact. Code stays open, no matter where it lands next.

Companies face real outcomes here. Imagine changing the Linux Kernel then selling devices using it - must those tweaks be shared? Usually yes, when handing out the altered software. Sharing comes into play because GPL v2 ties distribution to openness: send the program, share the blueprint too. That rule explains why Android gadgets come with published code. It is also why outfits such as Red Hat release their kernel work publicly. Still, one thing stands out: when a business uses a changed version of Linux on a server to deliver services, it does not have to share the source code, simply because distribution isn't happening - execution is. That gap, often called the Application Service Provider exception, became a target during the creation of GPL v3, even if the Linux Kernel itself sticks with version two.

One key thing stands out when comparing GPL and MIT licenses: they're built on very different ideas. A look at MIT shows it gives broad freedom right away. Using the code, changing it, sharing it - even inside closed software - all fits under its rules. Sharing your modifications? Not required here. Just make sure the original credit stays put; that's about the only rule. Freedom comes with barely any strings attached in this case. Unlike permissive licenses, GPL enforces openness through its rules. Derivative works must remain accessible to others. When using code under MIT terms, firms might fold it into proprietary software freely. With GPL, incorporating the code demands releasing modifications publicly - no exceptions allowed.

Should I ever create a project meant to stay truly open, my pick would be GPL. On first glance, MIT seems friendlier, yet down the line it lets others grab your effort, improve it, then close it off. That path gets blocked under GPL. True, certain companies shy away from its requirements. Still, when crafting core tools - say a kernel - the copyleft approach fits best. It carries the spirit of those who shaped it. That line between "free like a drink" versus "free like liberty" - Richard Stallman's pushed it for ages. Not just theory; things actually shift because of it. Getting

something at zero cost? That doesn’t mean you know what went into it, who brewed it, or if tomorrow’s batch stays the same. Real freedom with software hands power back to you. Peek under the hood. Tweak anything. Share your version. The Linux Kernel runs on that kind of open ground. Free to grab, yet better still - it's open for you to look at, tweak as needed. No waiting around for Linus Torvalds or the Linux Foundation when something breaks; spot a flaw, know how to mend it? Step in directly. Share that patch so others gain too.

A3. The Ethics of Open Source

Looking into open-source software means facing real ethical concerns instead of treating it like a perfect fix every time. Because programs now run vital areas - hospitals, banks, power grids - being clear about how they work turns into a way to demand responsibility. Code that anyone can see invites outside checks, spots flaws sooner, often pulls in more minds to help improve things. That explains why tools such as Linux Kernel or GCC earn deep trust across users everywhere.

Still, good reasons exist for keeping some software closed. Building programs takes effort, skill, money - resources that not every group can afford under an open model. Certain tools, like those used in defense or internal company operations, need secrecy to work properly. Pretending openness works everywhere overlooks how things actually function.

Figuring out how firms gain profit from software made by communities isn’t straightforward. Giants such as Google, Amazon, and Red Hat run deep on tools crafted mostly by unpaid helpers. Although some respond with donations or their own coding work, gains tend to cluster unevenly across groups. Yet one thing stands firm - the GPL sets clear rules. Its permission lets businesses apply and share the software whenever they stick to what it asks. Freedom comes guaranteed under that deal. Payment never does.

Someone once said we see further by standing on giants. Today’s code grows that way too. Not alone does any coder work - each piece connects to older pieces beneath it. With open source, this web shows clearly. People join in, adding atop what came before, skipping the need to rebuild everything anew.

Open source usually helps new ideas grow instead of holding them back. Take Android, Docker, or Kubernetes - big tools that came from shared communities.

Even though closed software exists, open models often push things forward faster. Progress speeds up most when people exchange insights and work together openly.

Part B — Linux Footprint

The shell scripts in this project were executed in a Unix-based macOS terminal environment, which closely mirrors Linux command-line behavior for these tasks; however, the technical discussion in this section refers specifically to standard Linux systems as required by the assignment.

Installation Methods

Usually people get the Linux Kernel by way of their operating system's update system. Distribution tools like apt for Debian or Ubuntu, dnf for Fedora or RHEL, handle delivery after testing ready-made builds. Building directly from code happens when coders run make, shaping the core to fit special needs or try new ideas. Embedded setups take a different path - compiled on one machine for another, then baked into fixed software packages meant for particular devices.

Key Directories

Inside Linux, the layout of core system parts follows a clear path across folders. Starting up needs the kernel file plus boot instructions, both sitting in /boot. When extra features load on demand, they come from kernel pieces kept under /lib/modules. Information about running processes appears live through special entries in /proc. Hardware details show up instantly thanks to /sys, another virtual space tied to active operations. Settings that shape behavior hide inside /etc as plain configuration files. Programs meant to talk with low-level functions wait in places such as /usr/bin or /sbin.

User Group Security

The core of Linux works in the most protected zone - known as Ring 0 - where it manages vital operations. Meanwhile, regular programs live outside that area, running separately in what's called user space. Because they're split apart like this, apps can't mess with critical system memory by accident - or on purpose. Files tied

to the kernel belong only to the root account, so normal users can’t alter them without special access. Only those with admin rights can make changes, which limits power to exactly who needs it.

Managing Services and Modules

Even though you do not run the kernel like a regular background task, its pieces can still be adjusted while running. Tools including lsmod, modprobe, rmmod, along with modinfo help handle those parts; meanwhile dmesg lets you see messages from the core system. Managing broader services falls to managers such as systemd, each building on what the kernel already offers.

Update Model

Not long after changes appear, they move through a clear pipeline. Starting in the mainline, code gets checked by developers who share feedback openly. Once approved, distro teams reshape those pieces for their systems. Testing follows, quietly but thoroughly. Fixes reach people fast - sometimes just a couple days post-discovery. Stability stays high because each step acts like a filter. Updates roll out via standard packages, no surprises.

Part C — The FOSS Ecosystem

What Linux Needs to Work

Out of many open-source parts, one stands out - GCC turns the Linux Kernel's C code into something machines understand. Built by GNU, it handles the heavy lifting when creating executable files. Running behind the scenes, Make steps in to organize each stage through configuration scripts. Instead of guessing what goes where, it follows rules written down in special instruction sheets. Then there’s Binutils, slipping in quietly with tools needed for connecting pieces together. Without these helpers, turning raw code into a working system would stall before it even started.

A single step beyond basic code, the standard C library - often called glibc - connects everyday programs to the core functions of the operating system. Though the kernel runs just fine without it, most software built for users leans heavily on this link. Without that connection, even simple tasks would stumble. Its role? Quiet

but essential in how things talk across layers.

What Linux Made Possible

Imagine how much the Linux Kernel shapes tech today. Most smartphones run Android, sitting atop a tweaked version of that core system. Think about cloud services - AWS, then Google Cloud, even Azure - they lean into Linux for handling virtual spaces, packaging apps, along with connecting systems smoothly. Every machine on the latest Top500 supercomputer list runs Linux - its adaptability and speed make it the clear choice. Though unseen, it powers everyday gadgets like routers and factory tools because compact, tweakable cores matter there.

The Lamp Stack Link

LAMP isn’t just a name - it stands for Linux, Apache, MySQL, then PHP - shaping how websites come alive. On top of Linux, things start moving: Apache pushes pages out, MySQL stores data tight, PHP builds what users see. From this mix came tools like WordPress, showing full systems can grow without closed code. What runs much online? Often, it began here.

Community and Governance

Down at the core, Linus Torvalds guides how changes flow into the Linux Kernel, while trusted leads handle specific parts like networking or memory management. Talk about new code happens out in the open, mainly over email threads anyone can see. Each patch moves forward only after feedback comes back from those responsible for each section. Work shows up first on the mailing list, where every opinion and debate sticks around for everyone to check later. No hidden talks - what gets said stays visible, shaping what eventually slips into the system.
From behind the scenes, work keeps moving because help arrives through resources, money, and shared planning. Big names such as Google, IBM, Intel, or even Microsoft add effort into development - yet control over how things evolve stays out of any one company's grasp.

Part D — Open Source vs Proprietary

The table below compares the Linux Kernel against Windows Server as its closest proprietary alternative in the server and infrastructure space. The comparison is honest — open source is not better in every dimension for every organization.

Dimension	Linux Kernel (Open Source)	Windows Server (Proprietary)
Cost	Free to download, use, and distribute. No licensing fees. Enterprises pay only for optional support services (e.g., Red Hat, Canonical).	Requires per-server or per-core licensing. Can cost thousands of dollars per deployment.
Security / Code Audit	Source code is publicly available and reviewed by thousands of developers worldwide. Vulnerabilities are often found and patched rapidly by the community.	Source code is proprietary. Security depends entirely on Microsoft's internal teams. Third-party audits are not independently possible.
Support & Reliability	Community support through mailing lists, Stack Overflow, and distributions. Enterprise support available commercially. Powers 90%+ of the world's servers.	Official Microsoft support with guaranteed SLAs. Familiar to many IT administrators. Some may find enterprise support simpler to procure.
Freedom to Modify	Full freedom to study, modify, and redistribute under GPL v2. Organizations can build custom kernel versions tailored to their hardware.	No access to source code. Modification is prohibited. Users depend entirely on Microsoft for feature additions.
Community vs Corporate Control	Governed by Linus Torvalds and a distributed maintainer network through the Linux Foundation. No single corporate entity controls the direction.	Controlled entirely by Microsoft. Product roadmap and update decisions are made internally.
Overall Verdict	Best choice for servers, cloud infrastructure, embedded systems, and research. Requires more technical expertise but offers unmatched flexibility and cost savings.	Easier onboarding for organizations already in the Microsoft ecosystem. Strong GUI tooling. Better suited for environments requiring vendor-managed support.

Verdict and Recommendation

Having gone through this audit in some depth, I would recommend the Linux Kernel for real-world deployment in most server, cloud, and infrastructure contexts. The combination of zero licensing cost, full source code transparency, a massive community of contributors, and decades of production reliability makes it a strong default choice for technical teams with the skills to manage it. The security model — where any researcher or organization can audit the code — is, in my view, a genuine advantage over closed systems, not just a philosophical preference. When a vulnerability is found in the Linux Kernel, the fix is typically developed and reviewed in the open, which means the community can verify its correctness.

That said, I would not recommend it uncritically. Organizations that rely on Windows-centric tooling, have no internal Linux expertise, or operate in environments where vendor-backed support agreements are required for compliance reasons may find that a proprietary stack is more practical for their situation. The Linux Kernel requires engineering investment to manage well. The ecosystem around it — distributions, support contracts, tooling — has matured significantly, but it is not zero effort.

As for contributing: yes, I would. Even as a student, submitting documentation improvements or bug reports is a meaningful contribution to a project used by billions of devices. The maintainer model means contributions are reviewed rigorously, which is itself educational. Understanding how the kernel development community works — the mailing list culture, the review process, the standard for commit messages — is professional knowledge worth having.

Shell Script Tasks

This section documents all five shell scripts written for this project. Each script includes the full source code, an explanation of its purpose, and a description of the shell concepts it demonstrates. The scripts were developed for a Unix-based terminal environment and are fully compatible with Linux systems as well.

Script 1 — System Identity Report

Purpose

This script generates a formatted welcome screen that displays key information about the running system. The intent is to introduce the Linux environment in the style of a startup audit log — showing who is logged in, what kernel they are running, how long the system has been up, and what license covers the underlying software.

Script Code:

```
#!/bin/bash
# Script 1 - System Identity Report
# Author: Abhishek Choudhary | Reg No: 24BCE11056
# Course: Open Source Software
# Description: Displays key system identity information.

STUDENT_NAME="Abhishek Choudhary"
REG_NO="24BCE11056"
SOFTWARE="Linux Kernel"

# Collect system info using command substitution
KERNEL=$(uname -r)
USERNAME=$(whoami)
HOME_DIR="$HOME"
UPTIME=$(uptime -p 2>/dev/null || uptime)
TODAY=$(date '+%d %B %Y, %H:%M:%S')

# Detect OS distribution across Linux and macOS
if [ -f /etc/os-release ]; then
    DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d= -f2- | tr -d '"')
elif command -v sw_vers >/dev/null 2>&1; then
    DISTRO="$(sw_vers -productName) $(sw_vers -productVersion)"
else
    DISTRO="Unknown System"
fi

LICENSE_MSG="GNU General Public License v2 (GPL v2)"
```

```
echo "==============================================="
echo "       OPEN SOURCE AUDIT - SYSTEM REPORT       "
echo "==============================================="
echo "Student Name   : $STUDENT_NAME"
echo "Registration No: $REG_NO"
echo "Software Chosen: $SOFTWARE"
echo "-----------------------------------------------"
echo "Distribution   : $DISTRO"
echo "Kernel Version : $KERNEL"
echo "Logged In User : $USERNAME"
echo "Home Directory : $HOME_DIR"
echo "System Uptime  : $UPTIME"
echo "Date and Time  : $TODAY"
echo "-----------------------------------------------"
echo "OS License     : $LICENSE_MSG"
echo "-----------------------------------------------"
echo "The Linux kernel is free software. Users are"
echo "allowed to use, study, modify, and share it"
echo "under the terms of the GPL v2 license."
echo "==============================================="
```

Output

```
(base) shreya@Abhisheks-MacBook-Air-4 OSS_Scripts % ./script1_system_identity_updated.sh
===============================================
       OPEN SOURCE AUDIT - SYSTEM REPORT
===============================================
Student Name   : Abhishek Choudhary
Registration No: 24BCE11056
Software Chosen: Linux Kernel
-----------------------------------------------
Distribution   : macOS 26.3.1
Kernel Version : 25.3.0
Logged In User : shreya
Home Directory : /Users/shreya
System Uptime  : 20:57  up 14 days, 21:13, 2 users, load averages: 1.29 2.51 3.44
Date and Time  : 31 March 2026, 20:57:04
-----------------------------------------------
OS License     : GNU General Public License v2 (GPL v2)
-----------------------------------------------
The Linux kernel is free software. Users are
allowed to use, study, modify, and share it
under the terms of the GPL v2 license.
===============================================
```

Script 2 — FOSS Package Inspector

Purpose

This script checks whether a specified open-source package is installed on the system and displays information about it. It adapts automatically to whichever package manager is available — dpkg on Debian-based systems, rpm on Red Hat systems, or Homebrew on macOS. After the package check, a case statement prints a short philosophical note about the selected package, connecting the technical tool to the broader open-source narrative.

Script Code

```
#!/bin/bash
# Script 2 - FOSS Package Inspector
# Author: Abhishek Choudhary | Reg No: 24BCE11056
# Course: Open Source Software
# Description: Checks whether a package is installed and prints details.

PACKAGE=${1:-"git"}

echo "==============================================="
echo "         FOSS PACKAGE INSPECTOR               "
echo "==============================================="
echo "Student Name   : Abhishek Choudhary"
echo "Registration No: 24BCE11056"
echo "Package Checked: $PACKAGE"
echo "-----------------------------------------------"

# Detect and use the appropriate package manager
if command -v dpkg >/dev/null 2>&1; then
    if dpkg -s "$PACKAGE" >/dev/null 2>&1; then
        echo "$PACKAGE is installed on this system."
        dpkg -s "$PACKAGE" | grep -E '^(Version|Maintainer|Description):'
    else
        echo "$PACKAGE is NOT installed."
        echo "You can try: sudo apt install $PACKAGE"
    fi
elif command -v rpm >/dev/null 2>&1; then
    if rpm -q "$PACKAGE" >/dev/null 2>&1; then
        echo "$PACKAGE is installed on this system."
        rpm -qi "$PACKAGE" | grep -E '^(Version|License|Summary)'
    else
        echo "$PACKAGE is NOT installed."
        echo "You can try: sudo dnf install $PACKAGE"
    fi
elif command -v brew >/dev/null 2>&1; then
    if brew list --formula | grep -qx "$PACKAGE"; then
        echo "$PACKAGE is installed through Homebrew."
        brew info "$PACKAGE" | head -n 8
```

```
    else
        echo "$PACKAGE is NOT installed through Homebrew."
        echo "You can try: brew install $PACKAGE"
    fi
else
    echo "No supported package manager found."
fi

echo ""
echo "----------------------------------------------"
echo "Software Note:"
echo "----------------------------------------------"

# Case statement: print philosophy note based on package name
case "$PACKAGE" in
    kernel|linux|linux-image*)
        echo "Linux Kernel: the open core of modern computing." ;;
    httpd|apache2|apache)
        echo "Apache HTTP Server: foundation of the open internet." ;;
    mysql|mariadb)
        echo "MySQL/MariaDB: open databases in millions of apps." ;;
    git)
        echo "Git: built by Torvalds when proprietary tools failed him." ;;
    vlc)
        echo "VLC: famous for openness and universal compatibility." ;;
    firefox)
        echo "Firefox: Mozilla's browser built for the open web." ;;
    python|python3)
        echo "Python: a community-shaped language for everyone." ;;
    libreoffice)
        echo "LibreOffice: a community-driven fork of OpenOffice." ;;
    *)
        echo "$PACKAGE is part of the wider FOSS ecosystem." ;;
esac

echo "==============================================="
```

Output:

```
 ==============================================
(base) shreya@Abhisheks-MacBook-Air-4 OSS_Scripts % ./script2_package_inspector_updated.sh git
 ==============================================
          FOSS PACKAGE INSPECTOR
 ==============================================
 Student Name   : Abhishek Choudhary
 Registration No: 24BCE11056
 Package Checked: git
 ----------------------------------------------
 git is NOT installed through Homebrew.
 You can try: brew install git

 ----------------------------------------------
 Software Note:
 ----------------------------------------------
 Git: version control built by Linus Torvalds when proprietary tools failed him.
 ==============================================
(base) shreya@Abhisheks-MacBook-Air-4 OSS_Scripts % ./script3_disk_auditor_updated.sh
 ================================================================
```

Script 3 — Disk and Permission Auditor

Purpose

This script performs a security-oriented audit of important system directories by reporting their permissions, ownership, and disk usage. It also checks the Linux kernel module directory for the currently running kernel, tying the general audit to the chosen software subject. Understanding directory permissions is fundamental to Linux system administration — misconfigured permissions on sensitive directories like /etc or /var/log can be a significant security risk.

Script Code

```
#!/bin/bash
# Script 3 - Disk and Permission Auditor
# Author: Abhishek Choudhary | Reg No: 24BCE11056
# Course: Open Source Software
# Description: Audits important directories for permissions, owner/group, and size.

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot")

echo "================================================================"
echo "                 DIRECTORY AUDIT REPORT                         "
echo "================================================================"
echo "Student Name   : Abhishek Choudhary"
echo "Registration No: 24BCE11056"
echo "----------------------------------------------------------------"
printf "%-15s %-25s %-12s\n" "Directory" "Permissions Owner/Group" "Size"
echo "----------------------------------------------------------------"
```

```
# For loop iterates through each directory in the list
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, group using awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        [ -z "$SIZE" ] && SIZE="Restricted"
        printf "%-15s %-25s %-12s\n" "$DIR" "$PERMS" "$SIZE"
    else
        printf "%-15s %-25s %-12s\n" "$DIR" "Not Found" "-"
    fi
done

echo ""
echo "---------------------------------------------------------------"
echo "Chosen Software Config/Module Check"
echo "---------------------------------------------------------------"

# Check kernel module directory for the running kernel version
KVER=$(uname -r)
KMOD_DIR="/lib/modules/$KVER"

if [ -d "$KMOD_DIR" ]; then
    KPERMS=$(ls -ld "$KMOD_DIR" | awk '{print $1, $3, $4}')
    KSIZE=$(du -sh "$KMOD_DIR" 2>/dev/null | cut -f1)
    [ -z "$KSIZE" ] && KSIZE="Restricted"
    echo "Path        : $KMOD_DIR"
    echo "Permissions : $KPERMS"
    echo "Size        : $KSIZE"
else
    echo "Kernel module directory not found for kernel $KVER"
fi

echo "================================================================"
```

Output:

```
==============================================
[(base) shreya@Abhisheks-MacBook-Air-4 OSS_Scripts % ./script3_disk_auditor_updated.sh
================================================================
                DIRECTORY AUDIT REPORT
================================================================
Student Name   : Abhishek Choudhary
Registration No: 24BCE11056
----------------------------------------------------------------
Directory       Permissions Owner/Group   Size
----------------------------------------------------------------
/etc            lrwxr-xr-x@ root wheel      0B
/var/log        drwxr-xr-x root wheel     107M
/home           lrwxr-xr-x root wheel       0B
/usr/bin        drwxr-xr-x root wheel      80M
/tmp            lrwxr-xr-x@ root wheel      0B
/boot           Not Found                 -

----------------------------------------------------------------
Chosen Software Config/Module Check
----------------------------------------------------------------
Kernel module directory not found for kernel version 25.3.0
================================================================
```

Script 4 — Log File Analyzer

Purpose

This script reads a log file and counts how many lines contain a specified keyword. It accepts both the file path and the keyword as command-line arguments, defaulting to 'error' if no keyword is given. The script includes input validation, a retry mechanism for empty files, and outputs the last five matching lines. Log analysis is a practical skill in Linux system administration — the kernel itself writes to system logs like /var/log/kern.log, and being able to search and summarize log content programmatically is genuinely useful.

Script Code

```
#!/bin/bash
# Script 4 - Log File Analyzer
# Author: Abhishek Choudhary | Reg No: 24BCE11056
# Course: Open Source Software
# Usage: ./script4_log_analyzer_updated.sh /path/to/logfile [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword if none provided
COUNT=0
```

```
ATTEMPT=1
MAX_ATTEMPTS=2

# Validate that a file path was provided
if [ -z "$LOGFILE" ]; then
    echo "Please provide a log file path."
    echo "Usage: $0 <logfile> [keyword]"
    exit 1
fi

# Check that the file actually exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found."
    exit 1
fi

# Retry loop (do-while style) if file is empty
while [ $ATTEMPT -le $MAX_ATTEMPTS ]; do
    if [ -s "$LOGFILE" ]; then
        break
    else
        echo "Attempt $ATTEMPT: File is empty. Checking again..."
        ATTEMPT=$((ATTEMPT + 1))
        sleep 1
    fi
done

if [ ! -s "$LOGFILE" ]; then
    echo "The file '$LOGFILE' is still empty after retrying."
    exit 1
fi

echo "================================================================"
echo "                    LOG FILE ANALYZER                           "
echo "================================================================"
echo "Student Name   : Abhishek Choudhary"
echo "Registration No: 24BCE11056"
echo "Log File       : $LOGFILE"
echo "Keyword        : $KEYWORD"
echo "----------------------------------------------------------------"

# While-read loop: process each line of the log file
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
echo "----------------------------------------------------------------"
```

```
grep -i "$KEYWORD" "$LOGFILE" | tail -5

echo "==============================================================="
```

Output:

```
(base) shreya@Abhisheks-MacBook-Air-4 OSS_Scripts % ./script4_log_analyzer_updated.sh /var/log/system.log log
 ==============================================================
                    LOG FILE ANALYZER
 ==============================================================
 Student Name   : Abhishek Choudhary
 Registration No: 24BCE11056
 Log File       : /var/log/system.log
 Keyword        : log
 --------------------------------------------------------------
 Keyword 'log' found 119 time(s) in /var/log/system.log

 Last 5 matching lines:
 --------------------------------------------------------------
 Mar 31 20:19:06 Abhisheks-MacBook-Air-4 login[12045]: USER_PROCESS: 12045 ttys010
 Mar 31 20:23:39 Abhisheks-MacBook-Air-4 syslogd[595]: ASL Sender Statistics
 Mar 31 20:33:47 Abhisheks-MacBook-Air-4 syslogd[595]: ASL Sender Statistics
 Mar 31 20:44:48 Abhisheks-MacBook-Air-4 syslogd[595]: ASL Sender Statistics
 Mar 31 20:55:08 Abhisheks-MacBook-Air-4 syslogd[595]: ASL Sender Statistics
 ==============================================================
```

Script 5 — Open Source Manifesto Generator

Purpose

This is the most interactive and personal of the five scripts. It asks the user three questions about their relationship with open-source software, then uses those answers to compose a personalised manifesto paragraph saved to a text file. The manifesto is then displayed in the terminal. The script captures the creative and philosophical spirit of the course — connecting the practical skills of shell scripting to the underlying values of open source.

Script Code

```
#!/bin/bash
# Script 5 - Open Source Manifesto Generator
# Author: Abhishek Choudhary | Reg No: 24BCE11056
# Course: Open Source Software
# Description: Asks three questions and generates a personalised manifesto file.

echo "==============================================================="
echo "            OPEN SOURCE MANIFESTO GENERATOR                    "
echo "==============================================================="
```

```
echo "Student Name   : Abhishek Choudhary"
echo "Registration No: 24BCE11056"
echo "-----------------------------------------------------------------"
echo "Answer the following three questions:"
echo ""

# Interactive input using read
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

TODAY=$(date '+%d %B %Y')
OUTFILE="manifesto_$(whoami).txt"

# Write manifesto to output file using output redirection (>)
echo "=================================================================" > "$OUTFILE"
echo "                 MY OPEN SOURCE MANIFESTO                      " >> "$OUTFILE"
echo "=================================================================" >> "$OUTFILE"
echo "Name           : Abhishek Choudhary" >> "$OUTFILE"
echo "Registration No: 24BCE11056" >> "$OUTFILE"
echo "Date           : $TODAY" >> "$OUTFILE"
echo "-----------------------------------------------------------------" >> "$OUTFILE"
echo "" >> "$OUTFILE"
echo "I use $TOOL every day, and that reminds me how powerful open" >> "$OUTFILE"
echo "source really is. For me, freedom means $FREEDOM, because I" >> "$OUTFILE"
echo "believe users should not just use software, but also have the" >> "$OUTFILE"
echo "right to study, improve, and share it. If I ever build $BUILD," >> "$OUTFILE"
echo "I would make it freely available so others can benefit too." >> "$OUTFILE"
echo "" >> "$OUTFILE"
echo "-- Abhishek Choudhary" >> "$OUTFILE"
echo "=================================================================" >> "$OUTFILE"

echo ""
echo "Manifesto saved to: $OUTFILE"
echo ""

# Alias concept (demonstrated as comment):
# alias manifesto='bash script5_manifesto_updated.sh'
# Running 'manifesto' would then launch this script directly.

cat "$OUTFILE"
```

Output

```
==============================================================
(base) shreya@Abhisheks-MacBook-Air-4 OSS_Scripts % ./script5_manifesto_updated.sh
==============================================================
            OPEN SOURCE MANIFESTO GENERATOR
==============================================================
Student Name   : Abhishek Choudhary
Registration No: 24BCE11056
--------------------------------------------------------------
Answer the following three questions:

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Choice
3. Name one thing you would build and share freely: A student productivity platform

Manifesto saved to: manifesto_shreya.txt

==============================================================
                 MY OPEN SOURCE MANIFESTO
==============================================================
Name           : Abhishek Choudhary
Registration No: 24BCE11056
Date           : 31 March 2026
--------------------------------------------------------------

I use Git every day, and that reminds me how powerful open
source really is. For me, freedom means Choice, because I
believe users should not just use software, but also have the
right to study, improve, and share it. If I ever build A student productivity platform,
I would like to make it freely available so others can learn
from it, improve it, and benefit from it just like I have
benefited from open-source software.

-- Abhishek Choudhary
==============================================================
```

Final Conclusion

This project set out to audit the Linux Kernel as an open-source software project — not just as a technical artifact, but as a product of specific historical circumstances, philosophical commitments, and community decisions. What emerged from that audit is a clearer picture of why the Linux Kernel matters beyond the fact that it happens to work well.

The origin story matters because it explains the values embedded in the project from the beginning. Linus Torvalds did not start Linux to build a business or to advance a political cause. He started it because he wanted something that did not exist and chose to share it when it became useful. That decision set a precedent: the code is available, the development is transparent, and contributions are welcome regardless of who you are or where you are from. Those values are not accidental — they are preserved by the GPL v2 license, which uses legal machinery to ensure that the code remains open through every generation of use and modification.

The ecosystem that has grown around the Linux Kernel is one of the most remarkable examples of collaborative knowledge-building in human history. Thousands of developers across dozens of countries have contributed to a codebase that powers the infrastructure of the modern world, from the servers that serve this document to the phone in someone's pocket. No single company or institution controls it, and that distributed ownership has proven to be a strength rather than a weakness.

The comparison with proprietary alternatives reveals trade-offs rather than a simple winner. Linux requires investment in expertise; Windows provides a more managed experience for organizations that need it. But for most infrastructure contexts, the combination of cost, transparency, flexibility, and community support makes Linux the more compelling long-term choice.

The five shell scripts written for this project are small demonstrations of a much larger truth: that open-source tools, when understood at the command-line level, give you genuine control over your computing environment. Being able to read a script, modify it, and understand what it does is the same kind of agency that the GPL guarantees at the software level. That connection — between the philosophy of free software and the practice of writing and sharing scripts — is what this course has been asking us to see.

The Linux Kernel is, in the end, proof that the open-source model works. Not as an ideology, but as a practical approach to building software that lasts.

References

[1] GNU Project. "The Free Software Definition." Free Software Foundation. Available at: https://www.gnu.org/philosophy/free-sw.html

[2] Open Source Initiative. "The Open Source Definition." Available at: https://opensource.org/osd

[3] Torvalds, L. (1991). "What would you like to see most in minix?" comp.os.minix newsgroup. Original post archived at: https://groups.google.com/g/comp.os.minix/c/dlNtH7RRrGA/m/SwRavCzVE7gJ

[4] Torvalds, L. & Diamond, D. (2001). Just for Fun: The Story of an Accidental Revolutionary. HarperBusiness.

[5] Raymond, E.S. "The Cathedral and the Bazaar." Available at: http://catb.org/~esr/writings/cathedral-bazaar

[6] The Linux Foundation. "About the Linux Foundation." Available at: https://www.linuxfoundation.org/about

[7] SPDX License List. "GPL-2.0-only." Available at: https://spdx.org/licenses/GPL-2.0-only.html

[8] Shotts, W. (2019). The Linux Command Line, 2nd Edition. No Starch Press. Freely available at: https://linuxcommand.org

[9] GNU Project. "GNU Bash Manual." Available at: https://www.gnu.org/software/bash/manual

[10] The Linux Kernel Archives. Available at: https://www.kernel.org

[11] Choose a License. "GPL v2." Available at: https://choosealicense.com/licenses/gpl-2.0

[12] Filesystem Hierarchy Standard (FHS). Linux Foundation. Available at: https://refspecs.linuxfoundation.org/fhs.shtml